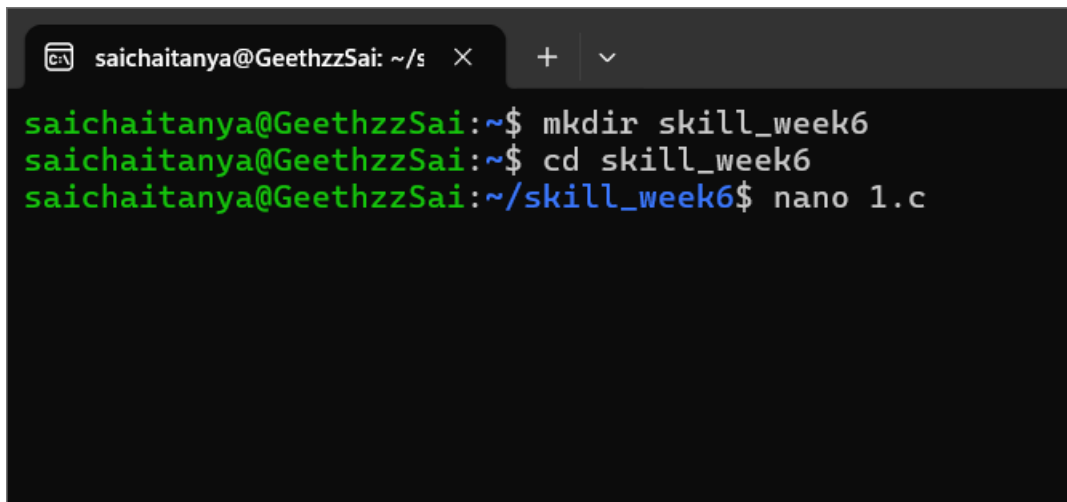


Name: M.Sai Chaitanya ID number: 2520090023

Week 6

SESSION-11

Task 1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.



```
saichaitanya@GeethzzSai: ~/s  ×  +  ∨  
saichaitanya@GeethzzSai:~$ mkdir skill_week6  
saichaitanya@GeethzzSai:~$ cd skill_week6  
saichaitanya@GeethzzSai:~/skill_week6$ nano 1.c
```

CODE:

```
GNU nano 8.7.1 1.c *
#include <stdio.h>
#include <string.h>

#define MAX 5
#define LEN 50

char history[MAX][LEN];
int count = 0;

void store(char *cmd) {
    if (count == MAX) { // Capacity limit
        for (int i = 1; i < MAX; i++)
            strcpy(history[i-1], history[i]);
        count--;
    }
    strcpy(history[count++], cmd);
}

void display() {
    for (int i = 0; i < count; i++)
        printf("%d: %s\n", i + 1, history[i]);
}

void retrieve(int n) {
    if (n >= 1 && n <= count)
        printf("Command: %s\n", history[n-1]);
    else
        printf("Invalid entry\n");
}

int main() {
    store("ls");
    store("pwd");
    store("cd Documents");
    store("gcc main.c");
    store("./a.out");
    store("clear"); // Oldest command removed
}
```

```
int main() {
    store("ls");
    store("pwd");
    store("cd Documents");
    store("gcc main.c");
    store("./a.out");
    store("clear");    // Oldest command removed

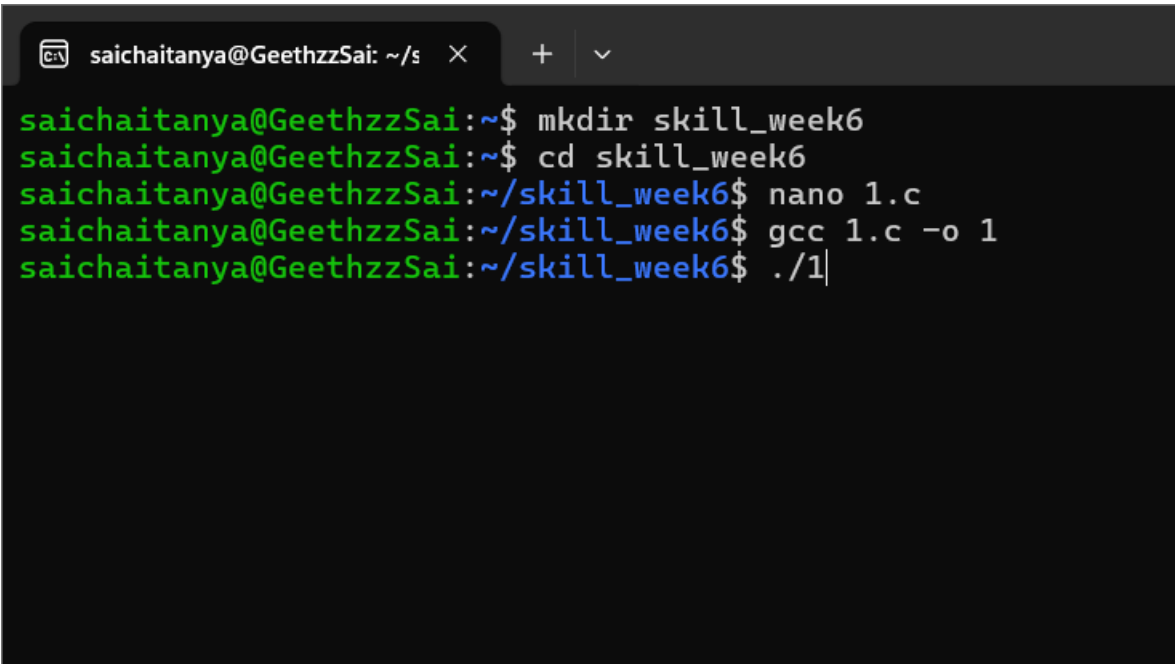
    printf("History:\n");
    display();

    retrieve(3);

    printf("Consistency: %s\n",
           count <= MAX ? "Valid" : "Invalid");

    return 0;
}
```

COMPILE AND RUN:



A terminal window with a dark background and light green text. The window title bar shows 'saichaitanya@GeethzzSai: ~/s' and a tab icon. The terminal content shows the following commands and their outputs:

```
saichaitanya@GeethzzSai:~$ mkdir skill_week6
saichaitanya@GeethzzSai:~$ cd skill_week6
saichaitanya@GeethzzSai:~/skill_week6$ nano 1.c
saichaitanya@GeethzzSai:~/skill_week6$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/skill_week6$ ./1
```

OUTPUT:

```
saichaitanya@GeethzzSai: ~/s  X + v
saichaitanya@GeethzzSai:~$ cd skill_week6
saichaitanya@GeethzzSai:~/skill_week6$ nano 1.c
saichaitanya@GeethzzSai:~/skill_week6$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/skill_week6$ ./1
History:
1: pwd
2: cd Documents
3: gcc main.c
4: ./a.out
5: clear
Command: gcc main.c
Consistency: Valid
saichaitanya@GeethzzSai:~/skill_week6$ |
```

Task 2: To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configuration.

CODE:

```
saichaitanya@GeethzzSai: ~/s  ×  +  v
GNU nano 8.7.1 2.c *
#include <stdio.h>
#include <string.h>

#define MAX 5

char cmd[MAX][30];
int n = 0;

void add(char *s) {
    strcpy(cmd[n++], s);
}

void show() {
    for (int i = 0; i < n; i++)
        printf("%s%s", cmd[i], i == n-1 ? "\n" : " | ");
}

int main() {
    add("ls");
    add("grep .c");
    add("sort");
    add("wc -l");

    printf("Pipeline: ");
    show();

    if (n > 0 && n <= MAX)
        printf("Pipeline layout: Valid\n");
    else
        printf("Pipeline layout: Invalid\n");

    printf("Execution order: ");
    for (int i = 0; i < n; i++)
        printf("%d ", i + 1);

    printf("\nConfiguration: Tested\n");
    return 0;
}
```

OUTPUT:

```
saichaitanya@GeethzzSai:~/skill_week6$  
saichaitanya@GeethzzSai:~/skill_week6$ nano 2.c  
saichaitanya@GeethzzSai:~/skill_week6$ gcc 2.c -o 2  
saichaitanya@GeethzzSai:~/skill_week6$ ./2  
Pipeline: ls | grep .c | sort | wc -l  
Pipeline layout: Valid  
Execution order: 1 2 3 4  
Configuration: Tested  
saichaitanya@GeethzzSai:~/skill_week6$ |
```

SESSION-12

Task 1: To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

CODE:

```
saichaitanya@GeethzzSai: ~/s  ×  +  ∨
GNU nano 8.7.1                                3.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int p[2];
    pipe(p);

    if (fork() == 0) {
        dup2(p[1], STDOUT_FILENO);
        close(p[0]); close(p[1]);
        execlp("ls", "ls", NULL);
    }

    if (fork() == 0) {
        dup2(p[0], STDIN_FILENO);
        close(p[0]); close(p[1]);
        execlp("wc", "wc", "-l", NULL);
    }

    close(p[0]); close(p[1]);
    wait(NULL);
    wait(NULL);

    printf("Data flow test completed.\n");
    return 0;
}
```

OUTPUT:

```
saichaitanya@GeethzzSai:~/skill_week6$
saichaitanya@GeethzzSai:~/skill_week6$ nano 3.c
saichaitanya@GeethzzSai:~/skill_week6$ gcc 3.c -o 3
saichaitanya@GeethzzSai:~/skill_week6$ ./3
6
Data flow test completed.
saichaitanya@GeethzzSai:~/skill_week6$ |
```

Task 2: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipeline.

CODE:

```
saichaitanya@GeethzzSai: ~/s × + v
GNU nano 8.7.1 4.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char *cmd[] = {"ls", "grep .c", "wc -l"};
    int p[2][2], n = 3;

    for (int i = 0; i < n - 1; i++)
        pipe(p[i]);

    for (int i = 0; i < n; i++) {
        if (fork() == 0) {
            if (i > 0) dup2(p[i-1][0], 0);
            if (i < n-1) dup2(p[i][1], 1);

            for (int j = 0; j < n-1; j++) {
                close(p[j][0]);
                close(p[j][1]);
            }

            execlp(cmd[i], cmd[i], NULL);
        }
    }

    for (int i = 0; i < n-1; i++) {
        close(p[i][0]);
        close(p[i][1]);
    }

    for (int i = 0; i < n; i++)
        wait(NULL);

    printf("Long pipeline completed.\n");
    return 0;
}
```

OUTPUT:

```
saichaitanya@GeethzzSai:~/skill_week6$  
saichaitanya@GeethzzSai:~/skill_week6$ nano 4.c  
saichaitanya@GeethzzSai:~/skill_week6$ gcc 4.c -o 4  
saichaitanya@GeethzzSai:~/skill_week6$ ./4  
Long pipeline completed.  
Long pipeline completed.  
saichaitanya@GeethzzSai:~/skill_week6$ |
```